

COMPUTATIONAL REPRODUCIBILITY

HOW CAN IT BE ACHIEVED IN PRACTICE?

Michael Hanke

Institute of Neuroscience and Medicine, Brain & Behavior (INM-7), Research Center Jülich

Institute of Systems Neuroscience, Medical Faculty, Heinrich Heine University Düsseldorf



@mih@mas.to

<http://psychoinformatics.de>



REPRODUCIBILITY: WHY SHOULD /CARE?



YOUR SCHOOLS, YOUR PENSION, YOUR COMFORT!

Reinhart and Rogoff's work showed average real economic growth slows (a 0.1% decline) when a country's debt rises to more than 90% of gross domestic product (GDP) – and this 90% figure was employed repeatedly in political arguments over high-profile austerity measures.

*The most serious was that, in their Excel spreadsheet, Reinhart and Rogoff had not selected the entire row when averaging growth figures: they omitted data from Australia, Austria, Belgium, Canada and **Denmark**.*

*When that error was corrected, the “**0.1% decline**” data became a **2.2% average increase** in economic growth.*



THE GOOD, THE BAD, AND THE UGLY

Traditionally, researchers have been taught to record every detail of their work, including experimental design, procedures, equipment, raw results, data processing, statistical methods and other tools used to analyse the results.

In contrast, relatively few researchers who employ computing in modern science [...] typically take such care in their work. In most cases, there is no record of workflow, hardware and software configuration, and often even the source code is no longer available (or has been revised numerous times since the study was conducted).

*We think this is a seriously lax environment in which **deliberate fraud** and **genuine error** can proliferate.*



If everything is important... **track everything!**
We can only track things that are written... **write down/out everything!**



OBJECTIVES FOR TODAY

- Reproduce a real paper, from empirical data to manuscript PDF
- Understand why and how that is doable

Dar, A. H., Wagner, A. S. & Hanke, M. (2020). REMoDNaV: Robust Eye-Movement Classification for Dynamic Stimulation. Behavior Research Methods, 53, 399–414. <https://doi.org/10.3758/s13428-020-01428-x>



CONCEPT OF THIS SESSION

- we can **only scratch the surface**
- **focus on doing**, not on theory and background
- **written materials are available**, presented slides shall only provide pointers and keywords
- **code-along**: everyone invited to run everything on their machines too
 - hands-on parts are (mostly) self-contained, if you get stuck in one, you can still try the next
 - open the slides in a browser and **copy&paste**, rather than type
 - if you need help: **ask sooner than later**, the answers are often insightful to everyone!



SWITCH TO HACKER MODE!

In 3-2-1



TERMINAL SURVIVAL GUIDE

Everywhere

But on Windows cmd.exe

Which directory am I in?

pwd

cd

Show (sub)directory tree structure

tree

tree

Change into a directory; parent directory

cd PATH; cd ..

cd PATH; cd ..

Go back into the HOME directory

cd

cd %userprofile%

List the content of a directory

ls [PATH]

dir [PATH]

Show the content of a file

cat PATH

type PATH

Print something to the terminal

echo "some text"

echo some text

Write something to a file

echo "some text" > PATH

echo some text > PATH



1ST TIME SETUP

- Open your terminal and run (please customize as needed!)
 - Verify installation, Should show version for `cmd:git` and `cmd:annex`

```
datalad wtf -S dependencies
```

- Open your terminal and run (please customize!):

```
git config --global user.name "FirstName LastName"  
git config --global user.email "email@example.com"
```

- Upgrade to latest DataLad versions

```
python -m pip install -U datalad datalad-container datalad-next
```

- Bring the future!

```
git config --global --add datalad.extensions.load next
```

Most of the hands-on parts are self-contained, and can be done individually, if this setup was successful!



WHAT IS DATALAD?

<http://datalad.org>



EXHAUSTIVE TRACKING OF RESEARCH COMPONENTS

Well-structured datasets (using community standards), and portable computational environments — and their evolution — are the precondition for reproducibility

```
# turn any directory into a dataset  
# with version control
```

```
% datalad create <directory>
```

```
# save a new state of a dataset with  
# file content of any size
```

```
% datalad save
```



CAPTURE COMPUTATIONAL PROVENANCE

Which data were needed at which version, as input into which code, running with what parameterization in which computational environment, to generate an outcome?

```
# execute any command and capture its output  
# while recording all input versions too  
  
% datalad run --input ... --output ... <command>
```



EXHAUSTIVE CAPTURE ENABLES PORTABILITY

Precise identification of data and computational environments, combined for provenance records form a comprehensive and portable data structure, capturing all aspects of an investigation.

```
# transfer data and metadata to other sites and services  
# with fine-grained access control for dataset components  
  
% datalad push --to <site-or-service>
```



REPRODUCIBILITY STRENGTHENS TRUST

Outcomes of computational transformations can be validated by authorized 3rd-parties. This enables audits, promotes accountability, and streamlines automated "upgrades" of outputs

```
# obtain dataset (initially only identity,  
# availability, and provenance metadata)
```

```
% datalad clone <url>
```

```
# immediately actionable provenance records  
# full abstraction of input data retrieval
```

```
% datalad rerun <commit|tag|range>
```



ULTIMATE GOAL: (RE)USABILITY

Verifiable, portable, self-contained data structures that track all aspects of an investigation exhaustively can be (re)used as modular components in larger contexts — propagating their traits

```
# declare a dependency on another dataset and  
# reuse it at particular state in a new context  
  
% datalad clone -d <superdataset> <url> <path-in-dataset>
```



HANDS-ON: DATALAD RDM ESSENTIALS

(this section is self-contained, only 1st-time setup needed)

Use a new terminal, or go back to your HOME directory!



DATASET CREATION

- Create a DataLad dataset

```
datalad create my1st  
cd my1st
```

- Run the command

```
gitk
```

and inspect the dataset history. Can you find

- the dataset identifier?
- the version label?
- the dataset creator?
- the dataset creation date?

- Run the command

```
gitk --all
```

What is the difference from before?



CONTENT TRACKING AND VERSION CONTROL

- Check the status of the dataset

```
datalad status
```

-> nothing to save, working tree clean

- Create a file inside the dataset (or copy one in with a file manager)

```
echo "exquisit" > myfile.txt
```

- Inspect the dataset status again

```
datalad status
```

- Save the dataset modification

```
datalad save -m "First time I ever datalad-saved"
```

- Run gitk again. Can you find
 - the file identifier?
 - the dataset modification date?



HANDS-ON: USE AND REUSE DATA MODULE

(this section is self-contained, only 1st-time setup needed)

Use a new terminal, or go back to your HOME directory!



CREATE A "YODA DATASET"

- Create a DataLad dataset, applying a specific post-creation routine

```
datalad create -c yoda iyoda  
cd iyoda
```

- Run the command

```
gitk
```

- what did that "YODA" setup actually do?
- how do we know that data module should go into inputs/?

- Run the command

```
datalad clone -d . https://github.com/datalad-handbook/iris_data inputs
```

- check the

```
datalad status
```

- what was actually saved in the iyoda dataset?

```
datalad subdatasets
```

- inspect the history of the "iris" dataset with gitk. When was it made, what does it contain? Hint: enter the dataset first



IMAGINE DATA SCIENCE!

- Create a script code/extract.py inside iyoda with content

```
from os.path import join as opj
import csv
with open(opj('inputs', 'iris.csv')) as csvfile:
    reader = csv.DictReader(csvfile)
    for row in reader:
        if row['variety'] != 'Setosa':
            continue
        print(row['petal.length'])
```

- careful: with Python consistent indentation with tabs OR spaces is necessary!
 - It will print all rows matching the Setosa variety
- Test the script

```
unix: python code/extract.py
```

```
win: python code\extract.py
```

- stop here and try figure out why it does not print any lines? (hint: the script above is fine as-is)
 - try again after running

```
datalad get inputs
```

- Save the script after confirming that it works

```
datalad save -m "Data extraction script" code
```



CAPTURE OUTPUTS

- Run the script and write its output into a file for further processing

```
unix: python code/extract.py > outputs.dat  
win: python code\extract.py > outputs.dat
```

- Check the status and save the dataset modification

```
datalad status  
datalad save -m "Create the desired setosa variety petal length data file"
```

- Use gitk to inspect the change record.
 - What information is captured?
 - Imagine yourself in a year: What information would you be missing?



CAPTURE PROVENANCE!

- Run the script again, but through DataLad, and declare inputs and outputs

- Unix:

```
datalad run -i inputs/iris.csv -o plength.txt "python code/extract.py > {outputs}"
```

- Windows:

```
datalad run -i inputs\iris.csv -o plength.txt "python code\extract.py > {outputs}"
```

- Use gitk to inspect the change record. What is different now?
- Note that `datalad-run` also has the `-m` option to set a message



ACTIONABLE PROVENANCE TO THE RESCUE!

- Force DataLad to loose the processed data

```
datalad drop --reckless availability plength.txt
```

- Verify that the file content is no longer around
- Re-execute the provenance record to recreate the file content

```
datalad rerun HEAD
```

- Check the file content! Check the dataset status

```
datalad status
```

What changed?



HANDS-ON: LET'S (START TO) REPRODUCE THE PAPER!

(this section is self-contained, only 1st-time setup needed)

Use a new terminal, and keep it open. This will run for while on its own!



ALL THERE IS TO REPRODUCE

There is one component still missing to explain. But it is under the hood, and we will get to that in a sec.

- Clone the dataset with the paper

```
datalad clone https://github.com/psychoinformatics-de/paper-remodnav.git
```

- Use a file manager to take a look at the content

- `main.tex`: the manuscript in LaTeX format
- `img`: all figures in SVG format this directory
- `results_def.tex`: all statistics reported in the paper

- Use DataLad to get an idea about the dataset setup

```
cd paper-remodnav  
datalad status --annex availability  
datalad subdatasets
```

- Cheat: save time, supply pre-downloaded 1.7G TAR files (optional)

```
git annex reinject --known --backend MD5E [PATH-TO]360338....tar  
git annex reinject --known --backend MD5E [PATH-TO]705094....tar
```

- Reproduce all figures and statistics reported in the paper

```
# first downloads and then processes 1GB of eye-tracking data  
datalad rerun results-containerized
```



BACK TO THE MAIN TRACK

- we created a (toy) data processing pipeline with Python
- we can run and rerun it
- we get the same results each time



BACK TO THE MAIN TRACK

- we created a (toy) data processing pipeline with Python
- we can run and rerun it
- we get the same results each time

Is that reproducibility?

Yes!

At least right here...

At least right now...



MAIN CHALLENGES OF (LONG-TERM) COMPUTATIONAL REPRODUCIBILITY

- not all software/algorithms produce the same results on all computers (even with identical input data)
- most software requires many other software components to work, it can be very difficult/impossible to recreate the exact same installer later/elsewhere
- hardware and software go out of fashion, expertise and availability go down over time



CONTAINERS! THEY HELP... A BIT... FOR NOW

- a **container** is a virtual computer that runs on real machine
- a container embeds an entire software environment, down to the operating system, with all software dependencies
- a container is stored in an **image** file (when it is not running), like the hard drive of a real computer, but in a file
- a container is built from a **recipe**, written and executable instructions to create a container image from scratch
- many container solutions exist (Singularity, Docker, podman, etc)
- a container will run wherever the respective container solutions run — an additional layer of insulation/interoperability buffer



HANDS-ON: CONTAINER AS AN APP

(this section is self-contained)

Use a new terminal, or go back to your HOME directory!



MOO IN A BOX

- Often a container is used to *package* an application
- Use Docker to obtain the rancher/cowsay app from the Docker Hub registry

```
docker pull rancher/cowsay
```

- Most of the time, a container can be executed with some input, and delivers some result. Try it!

```
docker run rancher/cowsay Moo!
```

- Docker is a service that can manage and run many containers. Check out these essential commands:

```
# what containers are running
docker ps
# what container images are available locally
docker images
# remove container images (when space runs out)
docker rmi ...
```



HANDS-ON: CONTAINER-BASED PYTHON INSTALLATION

(this section is NOT self-contained, but based on the iyoda session)

Use a new terminal, and/or cd into the iyoda dataset!



ALL THINGS END

- the Python script in `code/extract.py` was written for an old version of Python, version 2, which ran out of support in 2020.
- the code still runs with newer Python versions, but there is no guarantee that this will stay this way
- one can still install Python 2, but again no guarantee that this will remain possible
- not really a problem with a simple 8-line script, but imagine a PY2-based analysis toolkit comprising a few 10k lines of code
- a Python 2 in a container can postpone death. Let's try that



PYTHON 2 IN A BOX

- pull the container with Python 2 from Docker Hub

```
docker pull python:2-slim
```

- run it, and check that it is indeed a version 2.x

```
docker run python:2-slim python --version
```

- to be able to use it for data processing, we must give the container access to the data location on the host machine

```
unix: docker run -v $(pwd):/tmp -w /tmp python:2-slim python code/extract.py  
win:  docker run -v .:/tmp -w /tmp python:2-slim python code/extract.py
```

(slightly odd formatting of the windows command to better show how they are almost identical)

- -v ... exposes the current directory as /tmp *inside* the container
 - -w ... let's the given command run in /tmp inside the container
AKA the current working directory outside the container
- we can now use the containerized Python just like the system Python for processing data in this dataset



EXCURSION: CONTAINER CUSTOMIZATION

(this section is self-contained)

Use a new terminal, or go back to your HOME directory!



A NUMERICAL PYTHON 2

- containers are customized via *recipes* (called Dockerfiles in Docker)
- create a Dockerfile file in a mpy2 directory with this content

```
# base container
FROM python:2-slim
# code to run to customize it
RUN python -m pip install numpy
# command to run when the container is executed
CMD python
```

it extends the provided Python 2 container with the numpy package

- Build the container (run in the parent directory of mpy2)

```
docker build -t mpy2:latest mpy2
```

- Try running it

```
docker run -it mpy2:latest
```

- type

```
import numpy
```

and hit return, it should not error

- type `exit()` and hit return to stop the container



HANDS-ON: PROVENANCE CAPTURE WITH CONTAINERS

(this section is NOT self-contained, but based on the iyoda session)

Use a new terminal, and/or cd into the iyoda dataset!



REGISTER AND RUN CONTAINERS IN DATALAD DATASETS

- the datalad-container extension package provides this feature
- register a container from Docker Hub in the iyoda dataset

```
# 'container-python' is the name of the container in the dataset  
datalad containers-add -u dhub://python:2-slim container-python
```

- inspect the containers that the dataset knows

```
datalad containers-list
```

- run our extraction code via DataLad AND the container

Unix:

```
datalad containers-run -n container-python -i inputs/iris.csv -o plength.txt "python code/extract.py >  
plength.txt"
```

Windows:

```
datalad containers-run -n container-python -i inputs\iris.csv -o plength.txt "python code/extract.py >  
plength.txt"
```

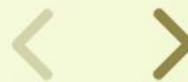
- inspect the provenance record with gitk
- find the container configuration defined in .datalad/config
- try a rerun

```
# may need 'master' instead of 'main'  
datalad rerun main
```



BUILDING BLOCKS OF COMPUTATIONAL REPRODUCIBILITY

- Track any digital information with DataLad datasets
 - text files
 - binary data
 - code
 - computational environments
- Nest DataLad datasets to **form and combine modular, reusable units** of information
- Capture computational provenance to not only track what was produced, but to also **record how it was done**
- Use **containers** as portal encapsulations of arbitrarily complex computing environments.



HANDS-ON: REPRODUCING A PAPER (PART 2)

(this section is NOT self-contained)

Go back to the terminal that executed `data1ad-rerun` on the paper!



INSPECT THE REPRODUCED PAPER

- Verify that everything was reproduced

```
datalad status  
gitk
```

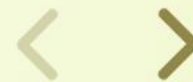
- Use gitk to find the three key pieces that enabled reproduction
 - the results-containerized tag, and the associated messages (notice the cryptographic signature)
 - the datalad-containers setup for docker-make (367bbeca)
 - the provenance record for building the container image itself (3dd49e36), and its recipe
- Build the manuscript PDF

```
datalad containers-run -n docker-make main.pdf
```



CRITICAL LOOK AT THE CONTAINER-APPROACH

- Ultimately suffers from the same problems as plain software use
 - if the container solution no longer installs/runs reproducing fails
 - not all container solutions are universally applicable/available
- It adds complexity to an already complex computational environment
- But
 - it can help keep things running today that would otherwise already fail today
 - it improves the portability of computational environments, also ongoing collaborations
 - receipes make containerized environment much more reproducible than real machines, this also helps for updating environment in ongoing projects (e.g., to fix software defects)
 - combined with version-control, containerization can be introduced in a later stage without loosing confidence



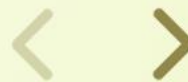
EXTENSIVE DOCUMENTATION AND TRAINING MATERIALS

<https://handbook.datalad.org> (or ISBN 979-8857037973)



DATALAD CONTACT AND MORE INFORMATION

Website + Demos	http://datalad.org
Documentation	http://handbook.datalad.org
Talks and tutorials	https://youtube.com/datalad
Development	http://github.com/datalad
Support	https://matrix.to/#/#datalad:matrix.org
Open data	http://datasets.datalad.org
Mastodon	@datalad@fosstodon.org
Twitter	@datalad



ADDITIONAL TOPICS



WHAT GOES INTO (DIFFERENT) DATA MODULES?

- Target audience is different
 - public vs. private
 - domain specific vs. domain general
- Pace of evolution is different
 - "factual" raw data vs. choices of (pre-)processing
 - completed acquisition vs. ongoing study
- Size impacts I/O and logistics
 - Git can struggle with 1M+ files
 - filesystems (licensing) can struggle with large numbers of inodes
- Legal/Access constraints
 - personal vs. anonymized data

